NOTE In the cloud native ecosystem, Tekton (https://tekton.dev) is a popular choice for defining deployment pipelines and other software workflows. It's an open source and Kubernetes-native solution hosted at the Continuous Delivery Foundation (https://cd.foundation). It runs directly on a cluster and lets you declare pipelines and tasks as Kubernetes custom resources.

GitHub Actions is a platform built into GitHub that lets you automate software workflows directly from your code repositories. A *workflow* is an automated process. We'll use workflows to model the commit stage of our deployment pipeline. Each workflow listens to specific *events* that trigger its execution.

Workflows should be defined in a .github/workflows folder in a GitHub repository root, and they should be described following the YAML format provided by GitHub Actions. In your Catalog Service project (catalog-service), create a commit-stage.yml file under a new .github/workflows folder. This workflow will be triggered whenever new code is pushed to the repository.

Listing 3.20 Defining a workflow name and trigger

```
name: Commit Stage      ❶
on: push                ❷
```

❶ The name of the workflow

❷ The workflow is triggered when new code is pushed to the repository.

Each workflow is organized into *jobs* that run in parallel. For now, we'll define a single job to collect the steps described previously in figure 3.8. Each job is executed on a *runner* instance, which is a server provided by GitHub. You can choose between Ubuntu, Windows, and macOS. For Catalog Service, we'll run everything on Ubuntu runners provided by GitHub. We'll also want to be specific regarding which *permissions* each job should have. The "Build and Test" job will need read access to the Git